장대한 · Unity 프로그래머

게임을 만드는 과정도 즐기는 프로그래머입니다.
Unity와 C#으로 입력, 아이템 배치, 상태 변화와 진행 규칙을 구현합니다.

플레이에서 생긴 문제를 코드로 풀어 가기

작은 조작 하나가 플레이의 재미로 이어지는 과정에 관심이 많습니다. 기획자·아티스트와 아이디어를 플레이로 옮긴 네 프로젝트를, 담당 역할과 설계 판단을 중심으로 정리했습니다.

01 Azit

비정형 인벤토리 · 점유 셀 검사와 이동 실패의 복원 경로

4인 / 프로그래밍 1명 · p.02

02 BOXXHOOTER

보스 패턴 · 공격 목록 선택과 개별 공격의 실행 분리

4인 / 프로그래밍 1명 · p.03

03 BoomBeat

8방향 리듬 · 곡별 노트 생성, 거리 판정과 노트 재사용

3인 / 프로그래밍 2명 중 1명 · p.04

04 THE KEYNG

계층 퍼즐 · 아래층 발판에서 바로 위층 캐릭터로 입력 전달

3인 / 프로그래밍 1명 · p.05

코드와 시연을 함께 확인할 수 있습니다

선별한 C# 9개, 사례별 README, 이미지와 짧은 플레이 영상을 GitHub에서 제공합니다. 각 사례의 **코드 읽기 시작점**에서 검토할 메서드를 찾을 수 있습니다.

GitHub에서 코드·프로젝트 설명 보기

코드는 원본 프로젝트 의존성이 있는 선별 자료이며, 이 저장소만으로 게임을 실행하는 구성은 아닙니다. 개발과 문서 정리에 AI 도구를 활용했습니다. 설명은 코드·기획 자료·시연을 대조한 범위로 작성했습니다.

Azit

비정형 아이템의 배치 검사와 실제 이동을 구분한 2D 탑다운 생존 RPG

4인 · 기획 1 / 그래픽 2 / 프로그래밍 1 | 내 역할: 프로그래밍 | Unity 2022.3.62f2 · C#



가방과 루팅 상자를 함께 다루는 실제 플레이 화면

그림 크기와 실제 점유 칸은 다르다

표시할 스프라이트와 점유 셀 목록인 Shape를 나눴습니다. 회전된 Shape의 각 셀에 기준 좌표를 더해, 사용 가능한 칸인지와 다른 아이템의 점유가 있는지 검사합니다. 화면이 달라도 같은 배치 규칙을 사용할 수 있습니다.

놓을 수 있는지 먼저, 옮기는 건 그다음

배치 검사는 ItemGridCore가 맡고, 원본 제거와 대상 배치는 ItemTransferTransaction이 처리합니다. TryMove는 원본 제거에 성공한 뒤 대상 배치를 요청하고, 대상 배치가 false를 반환하면 Rollback으로 원래 배치 복원을 시도합니다.

선택의 이유 배치 규칙을 바꿀 위치와 이동의 완료·실패 처리를 바꿀 위치를 구분하기 위해서입니다.

코드 읽기 시작점

src/azit/
ItemGridCore.cs · CanPlaceIgnoring (94행)
ItemTransferTransaction.cs · TryMove (37행), Rollback (112행)

실제 플레이 영상 보기 · GitHub 코드·프로젝트 설명 보기

약 7초 영상은 선반 조사·아이템 수집과 가방 변화를 보여 줍니다. 회전 배치와 실패 흐름은 코드에서 확인할 수 있습니다. 복원 콜백 자체가 실패할 가능성은 남아 있습니다.

BOXXHOOTER

공격 선택과 실행의 책임을 나눈 3D 쿼터뷰 액션 슈팅

4인 · 기획 1 / 그래픽 2 / 프로그래밍 1 | 내 역할: 프로그래밍 | Unity 2022.3.55f1 · C#



공격별 전조와 전투 장면 · 그래픽과 UI 제작은 팀이 분담

공격 목록과 공격 하나의 동작을 나누다

BossController가 체력에 맞는 패턴 풀과 다음 공격을 선택합니다. 개별 공격은 IBossPattern의 Begin·Tick·End라는 같은 호출 규칙으로 실행하도록 구성했습니다. 공격 목록을 고치는 부분과 공격 타이밍을 고치는 부분을 구분했습니다.

원형 공격의 전조에서 피해 생성까지

Pattern_RingCircles는 공격 위치를 계산한 뒤 전조를 표시합니다. 전조 콜백에서 연출을 만들고 attackDelay를 기다린 다음 피해 영역을 생성합니다. 전조 길이와 타격 지연을 따로 조절할 수 있습니다.

설계 시 고려할 점 공통 호출 규칙을 지키면서도, 패턴마다 종료 시점과 생성한 연출의 정리 책임을 관리해야 합니다.

코드 읽기 시작점

src/boxxhooter/
IBossPattern.cs → **BossController.cs** · RunPhaseRandomLoop (231행)
Pattern_RingCircles.cs · Run (67행), SpawnHitSequenceCo (159행)

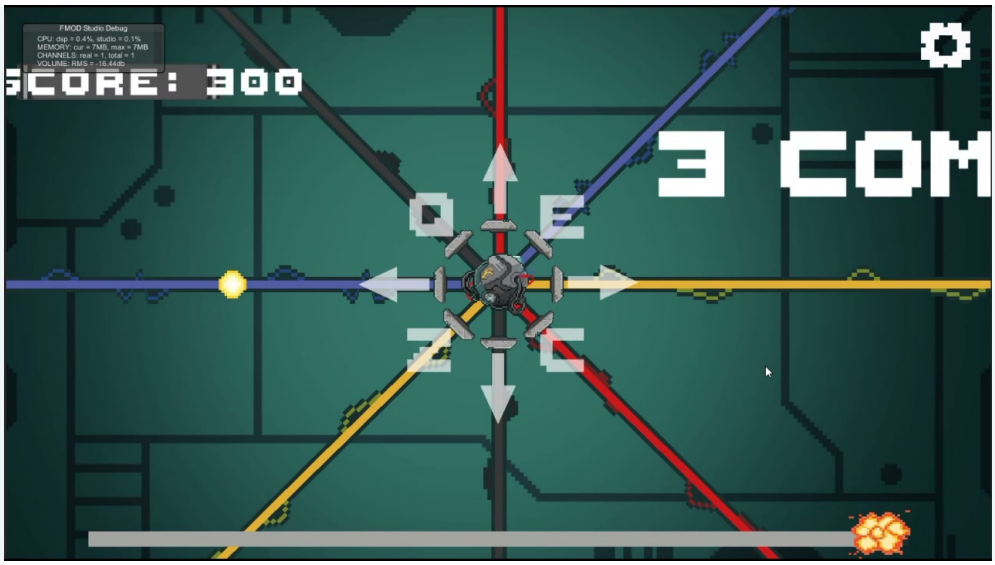
실제 플레이 영상 보기 · GitHub 코드·프로젝트 설명 보기

전조의 모양은 이미지에서, 실행 순서는 소스에서 확인합니다. 연결한 13초 영상은 별도의 개별 퍼즐 CLEAR와 전투 화면 복귀 장면입니다.

BoomBeat

노트 생성의 시계와 입력 판정의 기준을 구분한 8방향 리듬 게임

3인 · 프로그래밍 2명(기획 겸임 1명) / 그래픽 1명 | 내 역할: 프로그래밍 2명 중 1명



발표자료에 수록된 실제 플레이 · 노트 접근과 점수·콤보 표시

곡 데이터로 생성하고, 거리로 입력을 평가하다

Time.time - songStartTime을 생성 시계로 사용합니다. 곡 데이터의 도착 시각에서 noteArrivalOffset을 뺀 시점에 노트를 생성합니다. 일반 노트의 입력 결과는 판정점과 노트 사이의 거리를 Perfect·Good 허용값과 비교해 나눴습니다.

후보 목록과 처리 상태를 함께 확인하다

HitZone은 미처리 후보를 목록 순서로 검사하고, 하나를 적중시키면 이번 입력 검사를 끝냅니다. 구역을 벗어난 후보는 제거하며 아직 미처리 상태인 노트만 Miss로 처리합니다. 일반·롱 노트는 유형별 풀에 보관해 재사용합니다.

코드 읽기 시작점

src/boombeat/
NoteGenerator.cs · Start (21행), SpawnNotes (45행)
HitZone.cs · TryHitNote (71행), OnTriggerExit2D (49행)
NotePool.cs · GetNote (87행), ReturnNote (114행)

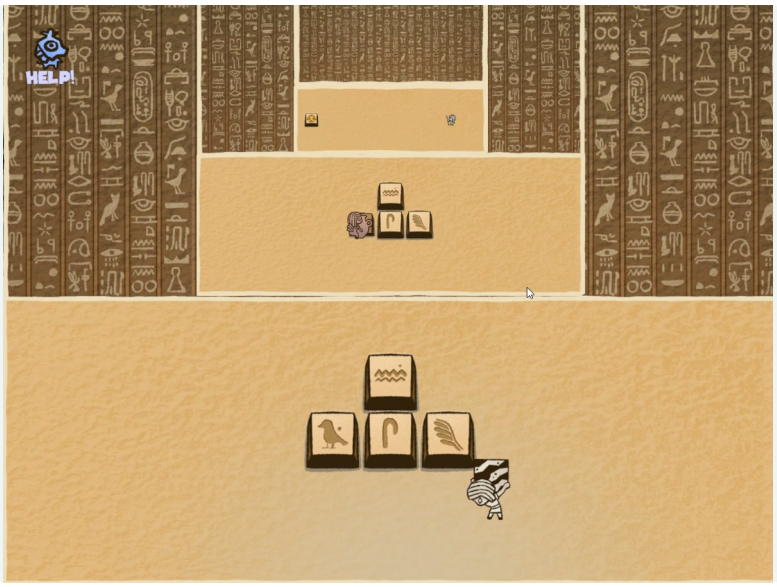
실제 플레이 영상 보기 · GitHub 코드·프로젝트 설명 보기

공개 범위는 노트 생성·이동, 거리 판정, 일반·롱 노트 재사용입니다. 같은 거리 허용값이라도 빠른 노트의 입력 시간 폭은 짧아집니다. 음악 출력 지연 보정과 속도별 동일한 시간 판정 폭은 다음 검토 대상입니다.

THE KEYNG

아래층의 방향 발판으로 바로 위층 캐릭터를 움직이는 2D 계층 퍼즐

3인 · 기획 1 / 그래픽 1 / 프로그래밍 1 | 내 역할: 프로그래밍 | Unity · C#



아래층의 발판과 위층 캐릭터를 연결하는 피라미드 구조

층 번호로 같은 조작 규칙을 연결하다

발판이 발동한 층과 방향을 전달하면 GameCoordinator가 바로 위층을 대상으로 정합니다. 제어 발판은 현재 층의 방향 발판과 다음 층을 열고, 최상층 제어 발판은 클리어로 연결합니다.

한 칸 이동과 유지 입력을 구분하다

한 칸 이동은 StepCommand로 전달합니다. 유지 입력은 오른쪽-왼쪽, 위-아래 상태를 벡터로 합쳐 SetRemoteDriveVector로 전달합니다. 반대 방향을 상쇄하고 길이가 1을 넘으면 정규화해 대각선 입력의 크기를 제한했습니다.

선택의 이유 층마다 연결 코드를 반복하는 대신, 발판 이벤트와 대상 층을 정하는 규칙을 나눠 같은 방식을 각 층에 적용하기 위해서입니다.

코드 읽기 시작점

src/the-keyng/**GameCoordinator.cs**
HandleCtrl (185행) · HandleArrowSingle (224행)
HoldState.ToVec (47행) · HandleArrowHoldChanged (233행)

실제 플레이 영상 보기 · GitHub 코드·프로젝트 설명 보기

21초 영상은 층 공개와 아래층 발판을 통한 위층 캐릭터 이동을 보여 줍니다. 발판·캐릭터 컴포넌트와 실제 층 연결은 원본 프로젝트에 포함된 의존성입니다.